```
                result->set_value(begin);                    <-- ❸
                done_flag->store(true);                      <-- ❹
                return;
            }
        }
    }
    catch(...)                                               <-- ❺
    {
        try
        {
            result->set_exception(std::current_exception());  <-- ❻
            done_flag->store(true);
        }
        catch(...)                                           <-- ❼
        {}
    }
  }
};
```

The main body of listing 8.9 is similar to the previous examples. This time, the work is done in the function call operator of the local `find_element` class ❶. This loops through the elements in the block it's been given, checking the flag at each step ❷. If a match is found, it sets the final result value in the promise ❸ and then sets the `done_flag` ❹ before returning.

If an exception is thrown, this is caught by the catchall handler ❺, and you try to store the exception in the promise ❻ before setting the `done_flag`. Setting the value on the promise might throw an exception if the promise is already set, so you catch and discard any exceptions that happen here ❼.

This means that if a thread calling `find_element` either finds a match or throws an exception, all other threads will see `done_flag` set and will stop. If multiple threads find a match or throw at the same time, they'll race to set the result in the promise. But this is a benign race condition; whichever succeeds is therefore nominally "first" and is therefore an acceptable result.

Back in the main `parallel_find` function itself, you have the promise ❽ and flag ❾ used to stop the search, both of which are passed in to the new threads along with the range to search ⓫. The main thread also uses